

APIs in Flutter

what is Json?-

Not a programming language-

هي طريقه تخزين الـ **Data** بشكل معين تفهمه كل لغات البرمجة، بتخزن الـ **Data** جوا الـ **Json** دي على شكل **List, Map**.

ازاي الـ **Data** دي بتتنقل؟

• **API = Application Programming interface**

هو الوسيط بين الـ **Flutter Front** والـ **backend**

عباره عن لينك بستخدمه علشان أعمل Request للـ **Data Base**:

← اجيب داتا | ← أبعث داتا | ← اعدل أو أمسح

* انا مش بتعامل مع الـ **Data** بشكل مباشر

الـ **backend** هما اللى بيتعاملوا مع الداتا والمفروض بيوفروا الـ **APIs** اللى انا أقدر استخدمها علشان أعمل الـ **Requests**

"مش هقول مثال المطعم"

Call flow → من أول ما تطلب **request** لحد ما يوصل للـ **Data Base**

Data flow → من أول ما الداتا تتحضر فى الـ **Data Base** وترجعك

Kind of API requests

• Get • Post • Patch • Delete

there are more but these our basic 4*

HTTP Requests Methods

- Retriving a single or multiple resources. → Get •
 - Creating a new resource. → Post •
 - Updating an existing resource. → Patch •
 - Updating or creating new resource. → Put •
 - Destroying a resource. → Delete •
 - Retrieve the resource's header. → Head •
 - Open a two-way socket connection to the remote server. → Connect •
 - Options •
 - describe the communication options for specified resource.
 - designed for diagnostic purposes during the develop. → Trace •
-

Components of API*

1. Baseurl
2. Endpoint
3. Headers
4. Body
5. Query parameters

* أول حاجه لو عاوزين نعمل SignIn

محتاجين نستدعى **Dio Package**

بعدين ناخد **object** من **Dio**

* فى الـ **SignIn** انا محتاج أعمل **Post request**

لأنى هبعث **Data** يعمل **Check** عليها ويرجعلي **respond**

```
signIn() {  
  Dio.post("link", {  
    data: { ... }  
  });  
}
```

==> **Post request** بتاعنا ده كده

|- عاوزين نخلي الكود نضيف

هنعمل حاجه اسمها **Api Consumer**:

هو ببساطه الجزء المسئول عن التعامل مع الـ **API**

بدل ما كل شاشة أو كل **Cubit** يكتب كود **Http request**

بنعمل **class** مخصوص بيقوم بشغله:

- بيعت الـ **Requests (Get - Post - Put - Delete)**
- بيستقبل الـ **Response**
- بيتعامل مع الايرورز
- بيضيف **headers** أو **token**. يحول البيانات لـ **JSON**

|- علاقة ده بالـ Repository

الترتيب غالبا بيكون:

UI → Cubit → Repository → API Consumer → API

- الـ **Cubit** يطلب تسجيل الدخول
 - الـ **Repository** يجهز البيانات
 - الـ **API Consumer** بيعت الـ **request** فعليا للسيرفر
-

* Dio Consumer : it's an implementation for API Consumer using Dio Package

Api Consumer → هو مجرد abstract class

Dio Consumer → الكلاس الحقيقي اللى بينفذ ال Request عن طريق Dio

"أي Request يتبع استخدام Dio"

* handle exceptions

التعامل مع الأخطاء اللى بتحصل أثناء ال API requests

بدل ما يحصل crash أو error غير مفهوم،

- فالمستخدم يقدر يشوف رساله مفهومه لو مثلا مفيش نت فتظهرله رساله بكنه

• ده برضه بيخلي الكود منظم أكثر

• ال Cubit يعرف يطلع Failure state صح

* Interceptor

هو حاجه بتقف فى النص بين ال **app** وال **API request/response**

فى أى **request** يروح أو **response** يرجع

ال **Interceptor** بيقدر:

• يعدل عليه • يضيف بيانات • يطلع **logs**

الاستخدامات:

1- إضافة ال **Token** تلقائياً بدل ما تضيف ال **token** فى كل **request** يدوي ال **interceptor** بيحطه مرة واحدة لكل ال **requests**

2- **Handle Errors**: ممكن تعرض رساله بال **error** أو تعمل عندها تلقائياً

3- **Logging**: يطبع ال **body** وال **response**

* **make debugging easier**

Headers :

هي معلومات إضافية تتبع مع ال **HTTP Request** لما ال **app** يكلم ال **api**
إحنا بنعتبره ظرف فى الطلب وال **headers** هى البيانات المكتوبه برا الظرف علشان السيرفر يفهم:

- نوع الداتا اللى جايا
- أى لغة ترجع بيها
- مين المستخدم
- الفورمات **JSON** ولا حاجه تانيه

Repository :

طبقه فى المشروع مسئولة عن جلب الداتا والتعامل معاها بدل ما ال **UI** أو ال **Cubit** يتعاملوا مباشرة

API/DataBase → UI/Bloc/Cubit يعنى هو الوسيط بين

- **Dartz**: library that provides functional programming tools and concepts, such as **Either**, **Option**, and immutable data handling to help developers.
-